

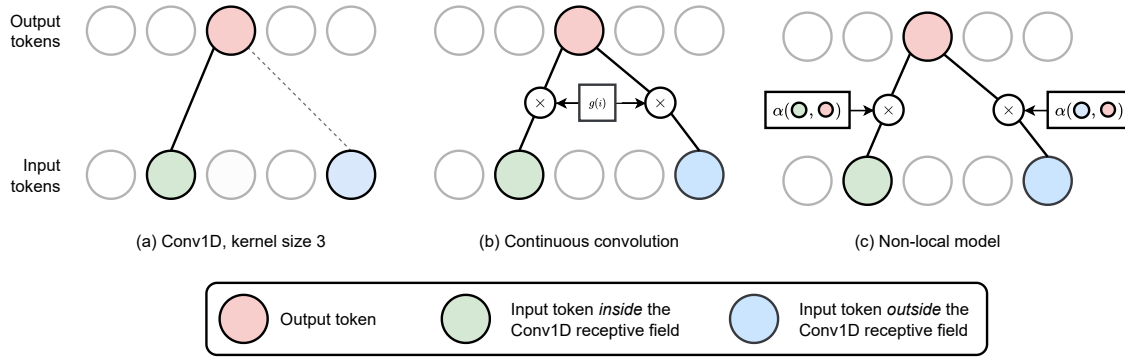


Figure E.10.1: Comparison between different types of convolution for a 1D sequence. We show how one output token (in red) interacts with two tokens, one inside the receptive field of the convolution (in green), and one outside (in blue). (a) In a standard convolution, the blue token is ignored because it is outside of the receptive field of the filter. (b) For a continuous convolution, both tokens are considered, and the resulting weight matrices are given by $g(-1)$ and $g(2)$ respectively. (c) In the non-local case, the weight matrices depend on a pairwise comparison between the tokens themselves.

we rewrite (E.10.1) as:

$$\mathbf{h}_i = \sum_{j=1}^n g(i-j) \mathbf{x}_j$$

↑
The sum is now on *all* tokens

This is called a **long convolution**, as the convolution spans the entire input matrix $\mathbf{X}$. It is also called a **continuous convolution** [RKG⁺22], because we can use $g(\cdot)$ to parameterize intermediate positions or variable resolutions [RKG⁺22]. The number of parameters in this case only depends on the parameters of g , while it does not depend on n , the length of the sequence. Defining g is non-trivial because it needs to output an entire weight matrix. We can recover a standard convolution easily:

$$g(i, j) = \begin{cases} \mathbf{w}_{i-j} & \text{if } |i-j| \leq k \\ 0 & \text{otherwise} \end{cases} \quad (\text{E.10.2})$$

This partially solves the problem of long-range dependencies, but it does not solve the problem of dependencies which are conditional on the input, since the weight given

to a token depends only on the relative offset with respect to the index i . However, this formulation provides a simple way to tackle this problem by letting the trained function g depend on the *content* of the tokens instead of their positions:

$$\mathbf{h}_i = \sum_{j=1}^n g(\mathbf{x}_i, \mathbf{x}_j) \mathbf{x}_j \quad (\text{E.10.3})$$

In the context of computer vision, these models are also called **non-local** networks [WGGH18]. We provide a comparison of standard convolutions, continuous convolutions, and non-local convolutions in Figure F.10.1.

10.1.2 The attention layer



The MHA layer is a simplification of (E.10.3). First, working with functions having matrix outputs is difficult, so we restrict the layer to work with scalar weights. In particular, a simple measure of similarity between tokens is their **dot-product**:

$$g(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^\top \mathbf{x}_j$$

As we will see, this results in an easily parallelizable algorithm for the entire sequence. For the following we consider a normalized version of the dot-product:

$$g(\mathbf{x}_i, \mathbf{x}_j) = \frac{1}{\sqrt{e}} \mathbf{x}_i^\top \mathbf{x}_j$$

This can be motivated as follows: if we assume $\mathbf{x}_i \sim \mathcal{N}(0, \sigma^2 \mathbf{I})$, the variance of each element of $\mathbf{x}_i^\top \mathbf{x}_j$ is σ^4 , hence the elements can easily grow very large in magnitude. The scaling factor ensures that the variance of the dot product remains bounded at σ^2 .

Because we are summing over a potentially variable number of tokens n , it is also helpful to include a normalization operation, such as a softmax:¹

$$\mathbf{h}_i = \sum_{j=1}^n \text{softmax}_j(g(\mathbf{x}_i, \mathbf{x}_j)) \mathbf{x}_j \quad (\text{E.10.4})$$

In this context, we refer to $g(\cdot, \cdot)$ as the **attention scoring function**, and to the output

¹The notation softmax_j in (E.10.4) means we are applying the softmax normalization to the set $\{g(\mathbf{x}_i, \mathbf{x}_j)\}_{j=1}^n$, independently for each i . This is easier to see in the vectorized case, described below.

of the softmax as the **attention scores**. Because of the normalization properties of the softmax, we can imagine that each token i has a certain amount of “attention” it can allocate across the other tokens: by increasing the budget on a token, the attention over the other tokens will necessarily decrease due to the denominator in the softmax.

If we use a “dot-product attention”, our g does not have trainable parameters. The idea of an attention layer is to recover them by adding trainable projections to the input before computing the previous equation. To this end, we define three trainable matrices $\mathbf{W}_k \sim (k, e)$, $\mathbf{W}_v \sim (v, e)$, $\mathbf{W}_q \sim (k, e)$, where k and v are hyper-parameters. Each token is projected using these three matrices, obtaining $3n$ tokens in total:

$$\text{Key tokens: } \mathbf{k}_i = \mathbf{W}_k \mathbf{x}_i \quad (\text{E.10.5})$$

$$\text{Value tokens: } \mathbf{v}_i = \mathbf{W}_v \mathbf{x}_i \quad (\text{E.10.6})$$

$$\text{Query tokens: } \mathbf{q}_i = \mathbf{W}_q \mathbf{x}_i \quad (\text{E.10.7})$$

These processed tokens are called the **keys**, the **values**, and the **queries** (you can ignore the choice of terminology for now; we will return on this point at the end of the section). The **self-attention** (SA) layer is obtained by combining the three projections (E.10.5)-(E.10.6)-(E.10.7) with (E.10.4):

$$\mathbf{h}_i = \sum_{j=1}^n \text{softmax}_j(g(\mathbf{q}_i, \mathbf{k}_j)) \mathbf{v}_j$$

Hence, we compute the updated representation of token i by comparing its query to all possible keys, and we use the normalized weights to combine the corresponding value tokens. Note that the dimensionality of keys and queries must be identical, while the dimensionality of the values can be different.

If we use the dot product, we can rewrite the operation of the SA layer compactly for all tokens. To this end, we define three matrices with the stack of all possible keys, queries, and values:

$$\mathbf{K} = \mathbf{XW}_k \quad (\text{E.10.8})$$

$$\mathbf{V} = \mathbf{XW}_v \quad (\text{E.10.9})$$

$$\mathbf{Q} = \mathbf{XW}_q \quad (\text{E.10.10})$$

The three matrices have shape (n, k) , (n, v) , and (n, k) respectively. As a side note, we can also implement them as a single matrix multiplication whose output is chunked in

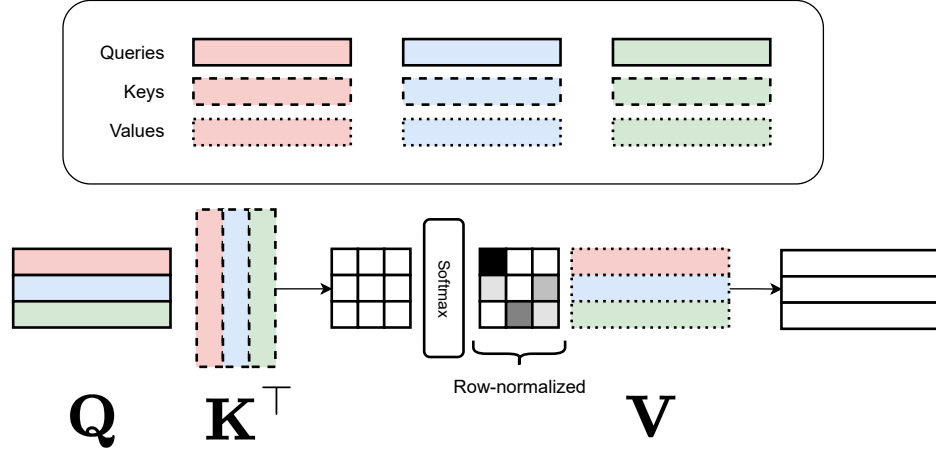


Figure F.10.2: Visualization of the main operations of the SA layer (excluding projections).

three parts:

$$\begin{bmatrix} K \\ V \\ Q \end{bmatrix} = X \begin{bmatrix} W_k \\ W_v \\ W_q \end{bmatrix}$$

The SA layer is then written as:

$$SA(X) = \text{softmax}\left(\frac{QK^T}{\sqrt{k}}\right)V$$

where we assume the softmax is applied row-wise. We can also make the projections explicit, as follows.



Important

Definition D.10.1 (Self-attention layer) The *self-attention* (SA) layer is defined for an input $X \sim (n, e)$ as:

$$SA(X) = \text{softmax}\left(\frac{XW_q W_k^T X^T}{\sqrt{k}}\right)XW_v \quad (\text{E.10.11})$$

The trainable parameters are $W_q \sim (k, e)$, $W_k \sim (k, e)$ and $W_v \sim (v, e)$, where k and v are hyper-parameters. Hence, there are $k(2e + v)$ trainable parameters, independent of n .

We show the operation of the layer visually in Figure F.10.2.

10.1.3 Multi-head attention

The previous layer is also called a **single-head** attention operation. It allows to model pairwise dependencies across tokens with high flexibility. However, in some cases we may have multiple sets of dependencies to consider: taking again the example of “*the cat, which belongs to my mother, is on the table*”, the dependencies between “*cat*” and “*table*” are different with respect to the dependencies between “*cat*” and “*mother*”, and we may want the layer to be able to model them separately.²

A multi-head layer achieves this by running multiple attention operations in parallel, each with its own set of trainable parameters, before aggregating the results with some pooling operation. To this end, we define a new hyper-parameter h , that we call the number of **heads** of the layer. We instantiate h separate projections for the tokens, for a total of $3hn$ tokens ($3n$ for each “head”):

$$\mathbf{K}_e = \mathbf{X}\mathbf{W}_{k,e} \quad (\text{E.10.12})$$

$$\mathbf{V}_e = \mathbf{X}\mathbf{W}_{v,e} \quad (\text{E.10.13})$$

$$\mathbf{Q}_e = \mathbf{X}\mathbf{W}_{q,e} \quad (\text{E.10.14})$$

$\mathbf{W}_{k,e}$ represents the key projection for the e -th head, and similarly for the other quantities. The **multi-head attention** (MHA) layer performs h separate SA operations, stacks the resulting output embeddings, and projects them a final time to the desired dimensionality:

$$\text{MHA}(\mathbf{X}) = \left[\text{softmax}\left(\frac{\mathbf{Q}_1\mathbf{K}_1^\top}{\sqrt{k}}\right)\mathbf{V}_1 \parallel \dots \parallel \text{softmax}\left(\frac{\mathbf{Q}_h\mathbf{K}_h^\top}{\sqrt{k}}\right)\mathbf{V}_h \right] \mathbf{W}_o \quad (\text{E.10.15})$$

Each SA operation returns a matrix of shape (n, v) . These h matrices are concatenated across the second dimension to obtain a matrix (n, hv) , which is then projected with a matrix $\mathbf{W}_o \sim (o, hv)$, where o is an additional hyper-parameter allowing flexibility in the choice of the output dimensionality.

²And everything depends on the cat, of course.

```

d = dict()
d["Simone"] = 2
d["Simone"]      # Returns 2
d["Smone "]      # Returns an error

```

Box C.10.1: A dictionary in Python: a value is returned only if a perfect key-query match is found. Otherwise, we get an error.

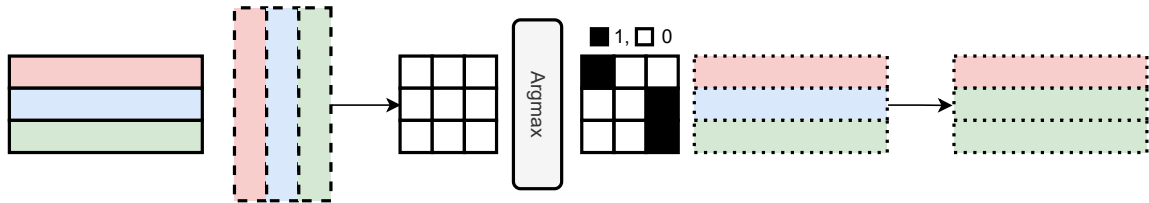


Figure F10.3: SA with a “hard” attention is equivalent to a vector-valued dictionary.

An explanation of the terminology



Discursive

In order to understand why the three tokens are called queries, keys, and values, we consider the analogy of a SA layer with a standard Python dictionary, which is shown in Box C.10.1.

Formally, a dictionary is a set of pairs of the form (key, value), where the key acts as an univocal ID to retrieve the corresponding value. For example, in the third and fourth line of Box C.10.1 we query the dictionary with two different strings (“Simone” and “Smone ”): the dictionary compares the query string to all keys which are stored inside, returning the corresponding value if a perfect match is found, an error otherwise.

Given a measure of similarity over pair of keys, we can consider a variant of a standard dictionary which always returns the value corresponding to the closest key found in the dictionary. If the keys, queries, and values are vectors, this dictionary variant is equivalent to our SA layer if we replace the softmax operation with an argmax over the tokens, as shown in Figure F10.3.

This “hard” variant of attention is difficult to implement because the gradients of the argmax operation are zero almost everywhere (we will cover discrete sampling and approximating the argmax operation with a discrete relaxation in the next volume). Hence, we can interpret the SA layer as a soft approximation in which each token is updated with a weighted combination of all values based on the corresponding key/query similarities.